케이녹 C언어 4회차 과제

17기 최동현 (202511688)

1-1. 포인터 (주소 연산자, 역참조 연산자)

- **포인터(Pointer)란?**

: C언어에서 **포인터**는 **'메모리의 주소'**를 **값**으로 가지는 **변수**입니다.

부가적으로 컴퓨터에서 사용하는 모든 데이터들은 **물리적 메모리(RAM)**에 저장되며, **고유한 메모리 주소(Address)**를 갖습니다. C언어는 **운영체제를 거치지 않고** 이 주소에 **직접 접근**할 수 있는 연산자를 제공합니다.

```c
// 포인터(Pointer) 선언 및 주소 대입
자료형* 포인터변수명 = 주소값(&); // 포인터 변수는 64비트 컴퓨터에서 어떤 자료형이든 8비트를 가짐

// 역참조(*)를 통한 데이터 조작
*포인터변수명 = 변경할_데이터값;

// 예시 코드
int a = 1;
int* aPtr = &a; // 정수형 변수 (a의 주소값) 대입
printf("%d\n", *aPtr); // aPtr가 가지고있는 주소가 가지고있는 값인 1출력
*aPtr = 100; // 역참조를 통해 a변수 값 100으로 조작
printf("%d\n", a); // 조작된 100값 출력
```

1-1. 포인터 (주소 연산자, 역참조 연산자)

- C언어 포인터 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main(void) {
    int a = 1;
    int* aPtr = &a;
    // 자료형이 어떻든 64비트 컴퓨터에서 포인터 변수는 전부 8바이트

    printf("a의 주소 : %p\n", (void*)&a);
    // %p를 사용한 a의 메모리 주소값 확인.
    // (void*)는 자료형이 없는 포인터 타입. 어떤 자료형의 주소든 담을 수 있음.
    printf("a의 주소 : %lx\n", (unsigned long)&a);
    // a의 주소를 부호없는 long타입으로 형변환하여 long type 16진수 출력.
    printf("aPtr이 가진 주소 : %p\n", aPtr); // (void*) 없이도 주소값 출력함.
    printf("aPtr의 주소 : %#lX\n\n", (unsigned long)&aPtr);
    // #은 0x 출력, X는 대문자 변환. (&)를 사용하여 aPtr의 주소값 출력.
    printf("수정 전 a의 값 : %d\n", *aPtr);

    *aPtr = 100;
    printf("수정 후 a의 값 : %d\n", a);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/main.c

a의 주소 : 0x7ff7bf315cf8
a의 주소 : 7ff7bf315cf8
aPtr이 가진 주소 : 0x7ff7bf315cf8
aPtr의 주소 : 0X7FF7BF315CF0

수정 전 a의 값 : 1
수정 후 a의 값 : 100

계속하려면 엔터 혹은 명령을 입력하십시오
```

1-2. 주소 개념 (변수, 함수, 문자열)

- **주소 개념 설명**

  : C언어에서 메모리에 로드 되는 모든 요소들은 **주소**를 갖습니다. 이 요소들이 저장되는 **메모리 세그먼트(Memory Segment)**들은 서로 다릅니다.

- **변수 주소: 스택(Stack)** 또는 **데이터/BSS**(**.data**/**.bss**) 영역에 존재하며 **읽고 쓰기**가 가능합니다.

- **문자열 주소:** 코드 내에 하드 코딩 된 **문자열 리터럴**(**"K-knock"** ← 문자열 데이터 그 자체)은 **읽기 전용 데이터**(**.rodata**) 영역에 저장됩니다.

- **함수 주소:** 컴파일 된 기계어 코드는 **텍스트/코드** 영역(**.text**)에 저장되며, **함수**의 **이름** 자체가 그 코드가 시작되는 메모리 주소를 의미합니다.

1-2. 주소 개념 (변수, 함수, 문자열)

- C언어 주소 개념 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int globalInit = 10; // .data 영역 (초기화 됨)
int globalNoInit; // .bss 영역 (초기화 안됨)

void function() {
    // 아무 기능도, 반환도 안하는 void 함수
}

int main(void) {
    int local = 10; // stack 영역
    const char* strPointer = "K-knock";
    // 상수 strPointer는 스택에 생성됨,
    // 하지만 그 안에는 .rodata에 있는 "K-knock" 문자열의 시작 주소가 담김

    // 변수 주소 ( 읽고 쓰기 가능 )
    printf("[.data] 초기화 된 전역변수 주소 : %p\n", &globalInit);
    printf("[.bss] 초기화 안 된 전역변수 주소 : %p\n", &globalNoInit);
    printf("[Stack] 지역변수 주소 : %p\n\n", &local);

    // 문자열 리터럴 주소 ( 읽기만 가능 )
    printf("[.rodata] \"K-knock\" 문자열 주소 : %p\n\n", strPointer);

    // 함수 주소 ( 읽기만 가능, 실행 코드 )
    printf("[.text] function 함수 시작 주소 : %p\n", function);
    printf("[.text] main 함수 시작 주소 : %p\n\n", main);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/main.c

[.data] 초기화 된 전역변수 주소 : 0x109f37000
[.bss] 초기화 안 된 전역변수 주소 : 0x109f37004
[Stack] 지역변수 주소 : 0x7ff7b5fc9c98

[.rodata] "K-knock" 문자열 주소 : 0x109f35618

[.text] function 함수 시작 주소 : 0x109f35560
[.text] main 함수 시작 주소 : 0x109f35570


계속하려면 엔터 혹은 명령을 입력하십시오
```

1-2. 주소 개념 (학습 중 의문점)

• C언어 포인터란? = 메모리 주소를 담는 변수

: 여기서 메모리 주소는 전부 **8바이트**의 **16진수**(0x0000...) 형태입니다. [그렇다면? 포인터 앞 자료형도 모두 똑같아야 하는 것 아닌가?] 라는 의문이 들었습니다.

(***포인터 변수 앞에 자료형을 붙이는***) 이유는 크게 **두 가지**가 있습니다.

1) 자료형이 붙는 이유는 **포인터가 가리키는 주소값**의 **자료형**이다.
   - [***int* p***] : "이 주소부터 시작해서 **4바이트** 읽어서 **정수**로 해석해라"
   - [***double* p***] : "이 주소부터 **8바이트** 읽어서 **실수**로 해석해라"
   - [***char* p***] : "이 주소부터 딱 **1바이트**만 읽어서 **문자**로 표현해라"

2) 어디까지 읽고 (**크기 결정**), 다음 칸은 어디인가? (**포인터 연산**)
   - 포인터에 [***+1***] 을 할 때, 자료형에 따라 **이동하는 거리**가 달라집니다.
   - **정수형(int)**일 경우 **4바이트** 만큼 이동, **문자형(char)**인 경우 **1바이트** 만큼 이동. (ex. **char* p = "test";**)
   - ***(p+1)**을 하게 되면 **test**의 두번째 값 **e**가 출력이 되고 **(void*)(p+1)**을 하게 되면 **값 e의 메모리 주소**가 출력이 됩니다.

결론) 자료형이 없다면, *컴퓨터는 메모리 주소지에 도착해도 어디까지 읽고 계산해야 하는지 모른다.*

1-3. 함수 (Call by Value, Call by Reference)

• 함수 (Call by Value와 Call by Reference)란?

: 두 방식 모두 **함수**에 **인자**를 **전달**하는 **방식**입니다.

1) Call by Value(값에 의한 호출) : 원본 데이터의 **'복사본'**을 스택 영역에 **새로 생성**하여 전달합니다. **함수 내부**에서 **값**을 **변경**해도 원본에는 **영향**을 **주지 않습니다**.

2) Call by Reference(참조에 의한 호출) : 원본 데이터가 위치한 **'메모리 주소'**를 **포인터**로 전달합니다. 함수 내부에서 **역참조(*)**를 통해 **원본**을 **수정**할 수 있으며, **배열**을 **인자**로 넘길 때는 기본적으로 **주소값**이 **전달**됩니다. 그러므로 *Call by Reference*처럼 동작을 하게 됩니다.

```
// Call by Value(값에 의한 호출) 정의 및 호출
반환형 함수명(자료형 매개변수) { ...; }
함수명(인자);

// Call by Reference(참조에 의한 호출) 정의 및 호출
반환형 함수명(자료형* 매개변수) { ...; }
함수명(&인자);
```

1-3. 함수 (Call by Value, Call by Reference)

- C언어 함수(Call By Reference) (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

void callByRef(int* val) {
    *val = *val + 1;
}

int main(void) {
    int num = 1;

    printf("변경 전 값 : %d\n", num);

    callByRef(val: &num);
    printf("변경 후 값 : %d\n", num);

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/main.c

변경 전 값 : 1
변경 후 값 : 2

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-1. 문자열 (null 및 선언 방식)

- **C언어에서 문자열(String)?**

  : C언어에서는 독립적인 **문자열(String)** 자료형이 **<u>존재하지 않습니다</u>**. 때문에 문자열은 **char 자료형**의 **배열**로 구현되며, 메모리 상 **문자(1바이트)들**이 **연속적으로 나열**된 형태입니다.

```
// 문자열 (자료형 배열로 선언)
char 배열명[크기] = "String";
char 배열명[크기] = { 'S','t','r','i','n','g','\0' };
// 중괄호를 사용하여 문자열을 초기화 한다면,
// 반드시 명시적으로 NULL(\0) 문자를 삽입해 줘야합니다. (문자열이 끝남을 알림)
```

2-1. 문자열 (null 및 선언 방식)

- **문자열(String) 선언방식**

  : C언어에서 문자열을 선언하는 방식은 **메모리**가 **할당**되는 **위치**와 **데이터**의 **수정 가능 여부**를 결정짓는 핵심 요소입니다.

```
// 배열 방식 (읽고 쓰기 가능 = Stack) :
//  1) 스택(Stack) 영역에 배열 크기만큼 메모리가 할당 됩니다.
//  2) 문자열 리터럴의 데이터가 스택으로 복사됩니다. 스택에 존재하므로 각 글자를 자유롭게 수정가능.
char 배열명[크기] = "초기화문자열";
배열명[0] = 's'; // 가능!

// 포인터 방식 (읽기만 가능 = Read-only Data[.rodata]) :
//  1) 문자열 리터럴 자체가 읽기 전용 영역(.rodata)에 생성됩니다.
//  2) 포인터 변수는 그 영역의 시작 주소만을 가리킵니다.
char* 포인터명 = "초기화문자열"; // 포인터명[0] = 's'; 할시 런타임 에러남 (읽기전용, 쓰기X)
const char* 포인터명 = "초기화문자열"; // const(상수)를 붙여 런타임 에러를 내지않게 가능.
```

2-2. 문자열 (배열, 문자열, 포인터 차이)

• 배열/문자열/포인터 물리적 차이

: 배열, 문자열, 포인터들의 **세가지 개념**의 **물리적 차이**가 있습니다.

1) **배열**: 고정된 크기의 연속된 **메모리 공간** 그 자체. (char arr[100]; -> sizeof(arr); // 100)

2) **문자열**: 메모리에 **나열 된 문자들**의 **집합**. 반드시 끝에 **NULL('₩0')**이 들어가야 **문자열 인정** 받습니다.

3) **포인터**: 메모리 **주소값**만을 **저장**하는 변수. 64비트 운영체제에서 **sizeof(포인터변수)**는 항상 **8바이트**입니다. 포인터 자체는 배열이 아니며, 배열의 주소를 가리키는 **리모컨 역할**만을 합니다.
(char* p = "K-knock"; -> sizeof(p); // 8)

2-3. 문자열 (문자열 표준 함수)

• 문자열 표준 함수

: C언어의 **<string.h>** 라이브러리로 **NULL 종료 문자열(₩0)**을 기반으로 동작하는 **표준 함수**들을 제공합니다.

1) strlen(**문자열**): **NULL 값** 만날 때까지 **문자 개수**(길이)를 반환

2) strcmp(**문자열1, 문자열2**): **두 문자열**의 메모리 값을 순차적으로 **비교** (같으면 0, 같지 않으면 0이 아닌 값)

3) strcpy(**목적지, 원본**): **원본 문자열**을 NULL 값이 나올 때까지 **목적지 주소**로 **복사**합니다.

4) strnlen(), strncmp(), strncpy(): 위와 기능은 동일하나 **버퍼 오버런(Buffer Overrun)**을 방지하기 위해 **N개**의 요소만 **제한**하여 기능을 실행시킵니다.

5) **그 외** 여러 **문자열 표준 함수**들을 지원합니다.

2-4. 문자열 (주의 사항!)

```c
#include <stdio.h>

int main(void) {
    char adminPassword[20] = "K-knock1234!"; // 관리자 패스워드 가정
    char userInput[4] = { [0]='A',[1]='B',[2]='C',[3]='D' };
    // {} 초기화 하는데 '\0'값 넣지않음

    printf("input address: %p\n", &userInput);
    printf("pw address: %p\n", &adminPassword);

    printf("입력값: %s\n", userInput);

    // 컴파일러가 최적화를 위해 패딩값 3바이트 집어 넣음 ('\0')
    for(int i = 5; i < 8; i++) {
        printf("패딩값%d: %c\n", (i-4), userInput[i]);
    }
    // 패딩 값 끝나고부터 adminPassword 부분 메모리 영역
    printf("관리자 패스워드: ");
    for(int i = 8; i < 20; i++) {
        printf("%c", userInput[i]);
    }
    printf("\n");

    return 0;
}
```

2-4. 문자열 (주의 사항!)

```
76%                                                  7%
~/Desktop/CHOI   vim ./KKnock/Day4/main.c

input address: 0x7ff7bc487c78
pw address: 0x7ff7bc487c80
입력값 : ABCD
패딩값 1:
패딩값 2:
패딩값 3:
관리자 패스워드 : K-knock1234!
계속하려면 엔터 혹은 명령을 입력하십시오
```

총 3바이트 차이

'₩0'을 넣지 않았으나 패딩값 '₩0'을 만나 ABCD만 출력

컴파일러가 패딩값을 넣지 않았다면 "ABCDK-knock1234!"가 출력됐을것임

★ 지금 배열 userInput 초기화시 **NULL('₩0')**이 들어가지 않았으나, 컴파일러가 알아서 **패딩값('₩0')**을 넣어 주었습니다. 만약 이러한 기능을 **지원하지 않는 컴파일러**라면 문자열 userInput 출력 시 **adminPassword '₩0' 값이 나올때까지 출력**되는 점 유의하셔야 합니다.

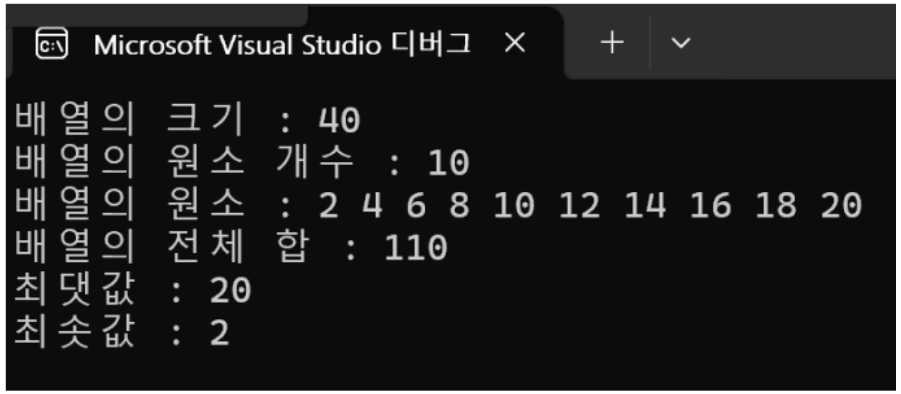
3. 실습 과제 (기본 배열 및 함수 다루기)

실습 1 - 기본 배열 및 함수 다루기

아래와 같이 출력하는 코드를 작성하시오.

조건

- 배열의 원소는 다음과 같다. { 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 }
- 배열의 전체 합, 최댓값, 최솟값을 구하는 사용자 지정 함수를 구현하여라.
- 배열의 크기와 배열의 원소 개수 값을 가지는 각각의 변수를 구현하여라.



3. 실습 과제 (기본 배열 및 함수 다루기)

```c
#include <stdio.h>

int sum(int arr[]) {
    int sum = 0;
    for(int i = 0; i < 40; i++) {
        if (arr[i] != 0) {
            sum += arr[i];
        }
    }
    return sum;
}

int max(int arr[]) {
    int max = arr[0];
    for(int i = 0; i < 40; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int min(int arr[]) {
    int min = arr[0];
    for(int i = 0; i < 40; i++) {
        if (arr[i] != 0 && arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}
```

```c
int main(void) {
    int arr[40] = { [0]=2,[1]=4,[2]=6,[3]=8,[4]=10,[5]=12,[6]=14,[7]=16,[8]=18,[9]=20 };
    int count = 0;

    printf("배열의 크기 : %lu\n", sizeof(arr) / sizeof(arr[0]));

    for(int i = 0; i < 40; i++) {
        if (arr[i] != 0) {
            count++;
        }
    }
    printf("배열의 원소 개수 : %d\n", count);

    printf("배열의 원소 : ");
    for(int i = 0; i < 40; i++) {
        if (arr[i] != 0) {
            printf("%d ", arr[i]);
        }
    }
    printf("\n");

    printf("배열의 전체 합 : %d\n", sum(arr));
    printf("최대값 : %d\n", max(arr));
    printf("최솟값 : %d\n", min(arr));

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/arrayFunction.c

배열의 크기 : 40
배열의 원소 개수 : 10
배열의 원소 : 2 4 6 8 10 12 14 16 18 20
배열의 전체 합 : 110
최대값 : 20
최솟값 : 2

계속하려면 엔터 혹은 명령을 입력하십시오
```

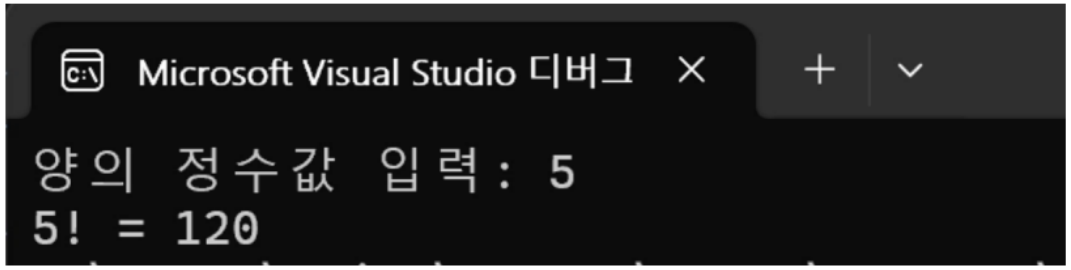
3. 실습 과제 (재귀함수 : 팩토리얼)

실습 2 - 재귀함수: 팩토리얼

재귀함수를 이용하여 팩토리얼을 계산하는 프로그램을 작성하시오.

팩토리얼 정의:
n! = n x (n-1) x (n-2) x ... x 2 x 1

조건:
- 재귀함수 Factorial(n)을 직접 구현
- 사용자로부터 자연수 n을 입력받기



3. 실습 과제 (재귀함수 : 팩토리얼)

```c
#include <stdio.h>

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n: n - 1);
}

int main(void) {
    int a = 0;

    printf("양의 정수값 입력 : ");
    scanf("%d", &a);

    printf("%d! = %d\n", a, factorial(n: a));

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/factorial.c

양의 정수값 입력 : 5
5! = 120

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (2차원 행렬 연산 덧/뺄셈)

과제 2 - 2차원 행렬 연산 (덧셈, 뺄셈)

행렬 덧셈과 뺄셈을 하는 프로그램을 작성해주세요.

행렬 입력, 출력, 덧셈, 뺄셈 모두 다 함수로 만들어주세요.

```
Microsoft Visual Studio 디버그

행렬의 행과 열 크기를 입력하세요 (최대 10): 2 2

첫 번째 행렬 입력:
행렬의 요소를 입력하세요:
matrix[0][0]: 1
matrix[0][1]: 2
matrix[1][0]: 3
matrix[1][1]: 4

두 번째 행렬 입력:
행렬의 요소를 입력하세요:
matrix[0][0]: 5
matrix[0][1]: 6
matrix[1][0]: 7
matrix[1][1]: 8

첫 번째 행렬:
1       2
3       4

두 번째 행렬:
5       6
7       8

행렬 덧셈 결과:
행렬:
6       8
10      12

행렬 뺄셈 결과:
행렬:
-4      -4
-4      -4
```

3. 실습 과제 (2차원 행렬 연산 덧/뺄셈)

```c
#include <stdio.h>
#define SIZE 10

void inputMatrix(int arr[SIZE][SIZE], int x, int y);
void printMatrix(int arr[SIZE][SIZE], int x, int y);
void sumMatrix(int arr1[SIZE][SIZE], int arr2[SIZE][SIZE], int result[SIZE][SIZE], int x, int y);
void subMatrix(int arr1[SIZE][SIZE], int arr2[SIZE][SIZE], int result[SIZE][SIZE], int x, int y);

int main(void) {
    int x = 0, y = 0;
    int arr1[SIZE][SIZE], arr2[SIZE][SIZE], sumMatrix1[SIZE][SIZE], subMatrix1[SIZE][SIZE];

    printf("행렬의 행과 열 크기를 입력하세요 (최대 10): ");
    scanf("%d %d", &x, &y);

    printf("첫 번째 행렬 입력: \n");
    printf("행렬의 요소를 입력하세요: \n");
    inputMatrix(arr: arr1, x, y);
    printf("\n");

    printf("두 번째 행렬 입력: \n");
    printf("행렬의 요소를 입력하세요: \n");
    inputMatrix(arr: arr2, x, y);
    printf("\n");

    printf("첫 번째 행렬: \n");
    printMatrix(arr: arr1, x, y);
    printf("\n");

    printf("두 번째 행렬: \n");
    printMatrix(arr: arr2, x, y);
    printf("\n");

    printf("행렬 덧셈 결과: \n");
    printf("행렬: \n");
    sumMatrix(arr1, arr2, result: sumMatrix1, x, y);
    printMatrix(arr: sumMatrix1, x, y);
```

```c
    printf("행렬 뺄셈 결과: \n");
    printf("행렬: \n");
    subMatrix(arr1, arr2, result: subMatrix1, x, y);
    printMatrix(arr: subMatrix1, x, y);


    return 0;
}

void inputMatrix(int arr[SIZE][SIZE], int x, int y) {
    for(int i = 0; i < x; i++) {
        for(int j = 0; j < y; j++) {
            printf("matrix[%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }
}

void printMatrix(int arr[SIZE][SIZE], int x, int y) {
    for(int i = 0; i < x; i++) {
        for(int j = 0; j < y; j++) {
            printf("%d\t",  arr[i][j]);
        }
        printf("\n");
    }
}

void sumMatrix(int arr1[SIZE][SIZE], int arr2[SIZE][SIZE], int result[SIZE][SIZE], int x, int y) {
    for(int i = 0; i < x; i++) {
        for(int j = 0; j < y; j++) {
            result[i][j] = arr1[i][j] + arr2[i][j];
        }
    }
}
```

```c
void subMatrix(int arr1[SIZE][SIZE], int arr2[SIZE][SIZE], int result[SIZE][SIZE], int x, int y) {
    for(int i = 0; i < x; i++) {
        for(int j = 0; j < y; j++) {
            result[i][j] = arr1[i][j] - arr2[i][j];
        }
    }
}
```

3. 실습 과제 (2차원 행렬 연산 덧/뺄셈)

```
78%                                                    7%
~/Desktop/CHOI   vim ./KKnock/Day4/2-dimensional-array.c

행렬의 행과 열 크기를 입력하세요 (최대 10): 2 2
첫 번째 행렬 입력:
행렬의 요소를 입력하세요:
matrix[0][0]: 1
matrix[0][1]: 2
matrix[1][0]: 3
matrix[1][1]: 4

두 번째 행렬 입력:
행렬의 요소를 입력하세요:
matrix[0][0]: 5
matrix[0][1]: 6
matrix[1][0]: 7
matrix[1][1]: 8
```

```
첫 번째 행렬:
1       2
3       4

두 번째 행렬:
5       6
7       8

행렬 덧셈 결과:
행렬:
6       8
10      12
행렬 뺄셈 결과:
행렬:
-4      -4
-4      -4

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (콜라츠 추측)

콜라츠 추측 (난이도: ⭐⭐)

사용자 지정 함수를 사용하여 콜라츠 수열을 배열에 저장 후 출력하는 코드를 작성하시오.

$$f(n) = \begin{cases} n/2 & \text{if } n \equiv 0 \pmod{2}, \\ 3n+1 & \text{if } n \equiv 1 \pmod{2}. \end{cases}$$

$$a_i = \begin{cases} n & \text{for } i = 0, \\ f(a_{i-1}) & \text{for } i > 0 \end{cases}$$

Microsoft Visual Studio 디버그

```
시작 정수 입력 : 6
콜라츠 수열 : 6 3 10 5 16 8 4 2 1
총 항 개수 : 9
```

3. 실습 과제 (콜라츠 추측)

```c
#include <stdio.h>

int count = 1;

int collatz(int n) {
    printf("%d ", n);
    if (n <= 1) return 1;
    n = (n % 2 == 0) ? (n / 2) : ((n * 3) + 1);
    count++;
    return collatz(n);
}

int main(void) {
    int a = 0;
    printf("시작 정수 입력 : ");
    scanf("%d", &a);
    collatz(n: a);
    printf("\n총 항 개수: %d", count);
    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/collatz.c

시작 정수 입력 : 6
6 3 10 5 16 8 4 2 1
총 항 개수: 9
계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (피보나치 수열)

재귀함수: 피보나치 (난이도: ⭐⭐)

정수 n을 입력받아, 재귀함수를 이용하여 피보나치 수열의 앞에서부터 n개의 값을 출력하는 프로그램을 작성하시오.

$$F_0 = 0$$
$$F_1 = 1$$
$$F_{n+2} = F_{n+1} + F_n$$

조건:

- 반드시 재귀함수를 사용하여 구현할 것

Microsoft Visual Studio 디버그

```
정수를 입력하세요: 8
피보나치 수열: 0 1 1 2 3 5 8 13
```

3. 실습 과제 (피보나치 수열)

```c
#include <stdio.h>

int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n: n - 1) + fibonacci(n: n - 2);
}

int main(void) {
    int a = 0;

    printf("정수를 입력하세요: ");
    scanf("%d", &a);

    printf("피보나치 수열: ");
    for(int i = 0; i < a; i++) {
        printf("%d ", fibonacci(n: i));
    }

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day4/fibonacci.c

정수를 입력하세요: 8
피보나치 수열: 0 1 1 2 3 5 8 13
계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (피보나치 수열 깊게 사고하기)

> 재귀 피보나치 함수 깊게 생각해보기 (난이도: ⭐⭐⭐)

위 점화식을 활용해서 작성한 재귀 피보나치 함수로 `fibo(40)` 을 계산하면 시간이 좀 걸리는 모습을 확인할 수 있을 것입니다.

- 왜 느릴까요?
- `fib(5)` 를 계산하는 동안 `fib(2)` 는 몇 번 호출될까요?
- 이 문제를 해결하기 위한 기법이 있을까요? (재귀함수 형태는 그대로 유지합니다)

Q1) 왜 느릴까요?

A1) return fib(n – 1) + fib(n – 2) 형식으로 계속 가지 치면서 재귀하는 함수 당 또 다른 함수가 2개씩 늘어나버리니 이 가지가 너무 커버리면 컴퓨터가 연산해야 할 양이 너무 많아지기 때문입니다.

Q2) fib(5)동안 fib(2)는 몇 번 호출?

A2) | fib(5) = fib(4) + fib(3) | fib(4) = fib(3) + fib(2) | fib(3) = fib(2) + fib(1) | fib(2) = fib(1) + fib(0) | 으로 총 3번 호출됩니다.

3. 실습 과제 (피보나치 수열 깊게 사고하기)

Q3) 이 문제를 해결하기 위한 기업이 있을까요? (재귀함수 형태는 유지)
A3) 어떻게든 컴퓨터의 연산량을 줄인다! (계속 재귀하여 연산을 다 돌리지 말고, 연산했던 값을 기록해보자)

```
// 전에 연산했던 값을 기록하기 위한 수열저장 배열 선언
int arr[1000] = {0};

// 피보나치 함수 수정
int fib(int n) {
        // fib(0)이면 0반환, fib(1)이면 1반환
        if(arr[n] != 0) {     // arr[n]이 0이 아니라는 말은 즉, 계산한적이 있는 값.
                return arr[n];   // 더 이상 연산하지 않고 배열에 있는 값 반환.
        }
        arr[n] = fib(n – 1) + fib(n – 2);    // 배열에 값이 없으면 연산하여 배열(데이터베이스)에 저장.
        return arr[n];    // 그리고 해당 피보나치 값을 반환.
}
```

A3) 이렇게 되면 fib(40)을 한번하고 다시 한번 하게 되면 배열(데이터베이스)에 이미 수열이 저장되어 있기에 2번째 실행에선 더욱 빠른 모습을 볼 수 있습니다. (야호~!)

감사합니다

17기 최동현